Assignment 8
DATA 260

GitHub repository: https://github.com/SaminaMaraj/data260-4098 (tag hw8)

Name: Kazi Samina Maraj Mumu

Email: Kazisaminamaraj.mumu@sjsu.edu

Date: 10/29/2026

Personal Configuration and Project Information

SID4	PORT_BASE	PREFIX	SEED	VERIFY_SEED	DOMAIN_ID
4098	8498	s4098	4098	264098	2

Item	Value
Domain	Municipal transit incidents (primary entity: Incident, so CREATE_INCIDENT, GET_INCIDENT and incident_id)
Hardware	Apple MacBook Pro, M5 chip, 24 GB RAM
Local model	qwen3:8b via Ollama (Part 2 agents and the Part 3 consumer-death runs)
Kafka	apache/kafka:3.9.2, one KRaft broker in Docker; Python client aiokafka 0.12.0; Python 3.12
Verified commit	d1c53f9 (code + all results; make verify-hw08 ran on this commit, 18/18)
Tagged commit (hw8)	The commit tagged hw8 comes right after d1c53f9 and only adds report files (this PDF, METRICS.md, AI_USE.md, verification.json). A PDF cannot contain the hash of the commit that adds it, so the tag hash is in my Canvas comment (git rev-parse 'refs/tags/hw8^{commit}').

I kept extending the same transit incident repository from HW1-HW7 and did not make a separate copy of the code. The new code is in shared root-level folders: `src/hw8_kafka/` (Kafka code, worker, API wrapper, agent loop), `code/hw8/` (Planner, Writer, Reviewer, send_question.py and the Part 3 test runner), `homework_demo.py` and `docker-compose.hw8.yml`. All evidence is in `reports/hw08/` (RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, verification.json).

Part 1 - Backend Service Request/Response over Kafka

I. Setup: topics and containers

Kafka runs as one broker in Docker. I turned off automatic topic creation in the broker, so every topic has to be created on purpose with the right name. I created all seven PREFIX topics with the Kafka CLI (code/hw8/create_topics.sh). For Part 1 only two of them are used: `s4098_entity_requests` and `s4098_entity_responses`.

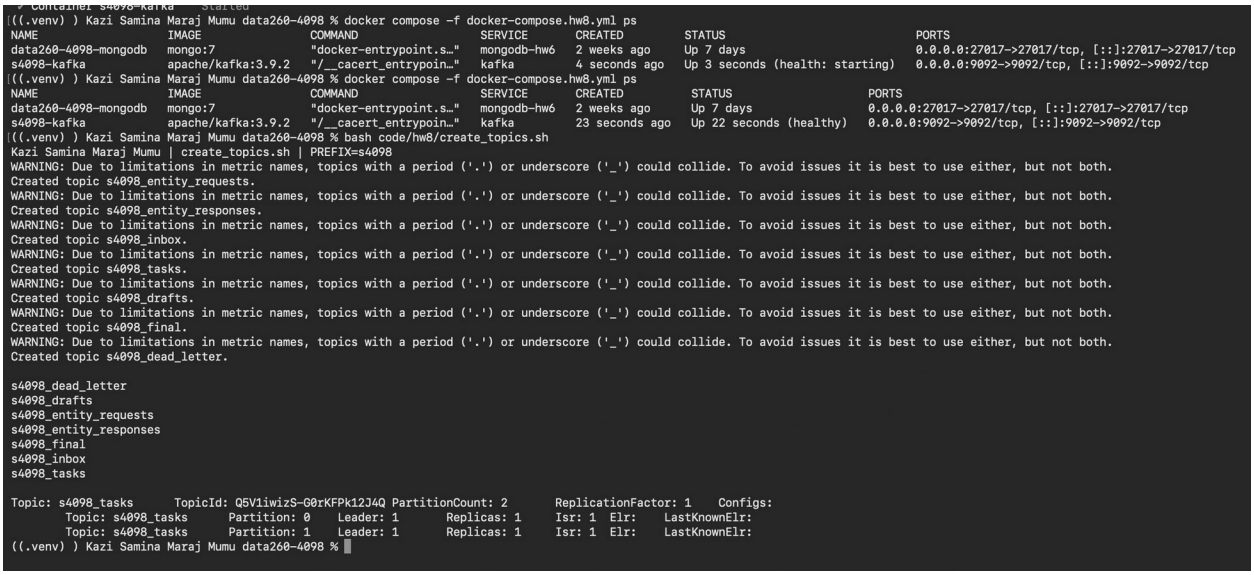


Figure 1. Kafka broker (s4098-kafka) is healthy and the seven s4098_ topics are created with the Kafka CLI. s4098_tasks has 2 partitions for the Part 3 ordering test.

The worker and the FastAPI wrapper are built from the same small image and run as two separate containers next to Kafka. The API is published on my PORT_BASE 8498.

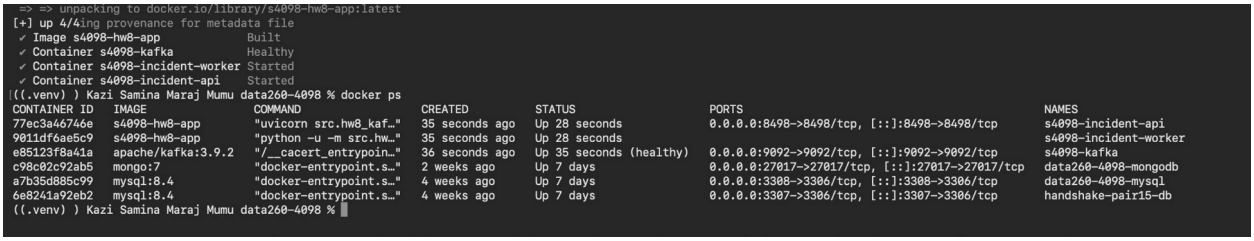


Figure 2. docker ps: Kafka broker (s4098-kafka), Part 1 worker (s4098-incident-worker) and FastAPI wrapper (s4098-incident-api, port 8498) running as separate containers.

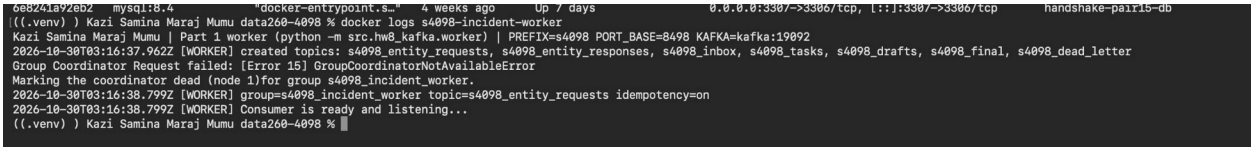


Figure 3. Worker container log: "Consumer is ready and listening..." on s4098_entity_requests in group s4098_incident_worker. The GroupCoordinatorNotAvailable line is a normal one-time retry while the broker finishes starting.

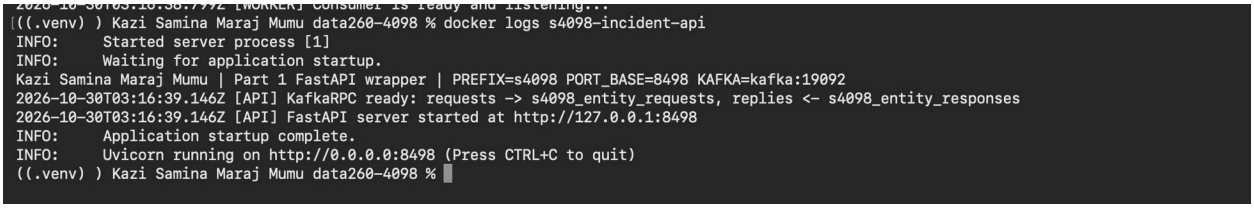


Figure 4. FastAPI container log: KafkaRPC connected (requests -> s4098_entity_requests, replies <- s4098_entity_responses) and "FastAPI server started at http://127.0.0.1:8498".

II. Topics and message schemas

Topic	Who sends -> who reads
s4098_entity_requests	client / FastAPI wrapper -> worker (group s4098_incident_worker)
s4098_entity_responses	worker -> FastAPI wrapper (this is the topic named in reply_to)

Request message (s4098_entity_requests)

```
{ "correlation_id": "<uuid, created once by the client>",
  "reply_to": "s4098_entity_responses",
  "operation": "CREATE_INCIDENT",            or "GET_INCIDENT"
  "data": { "incidentTitle", "routeLine", "submitterEmail", "description",
            "category", "termsAccepted", "passengersAffected" }   or { "incident_id" } }
```

Response message (s4098_entity_responses)

```
{ "correlation_id": "<same value>", "success": true, "operation": "CREATE_INCIDENT",
  "incident_id": "<uuid4>", "message": "Incident created" }
{ "correlation_id": "<same value>", "success": true, "operation": "GET_INCIDENT",
  "incident": { "incident_id": "...", ...all incident fields..., "submissionDate":
"..." } }
{ "correlation_id": "<same value>", "success": false, "error": "Incident not found" }
```

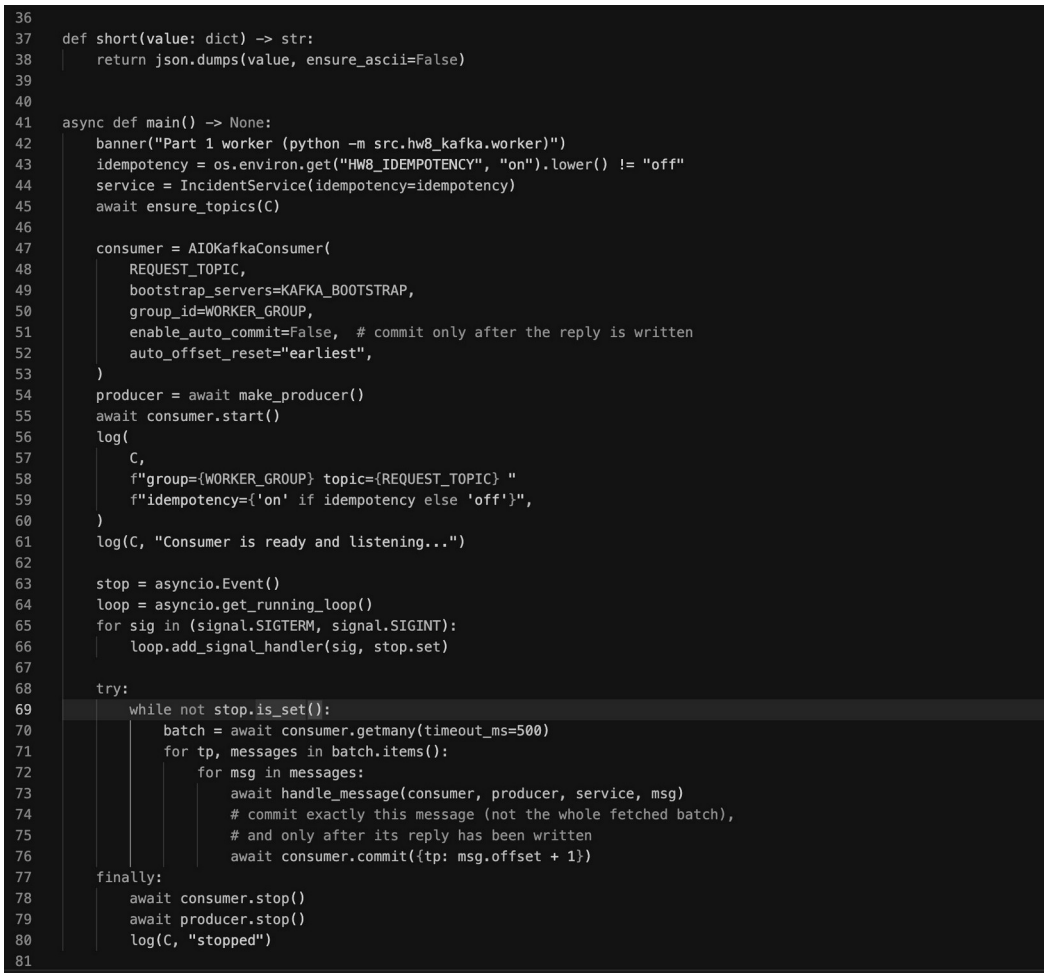
The worker keeps the incidents in a plain Python dictionary. It checks the data with the same rules as my HW4 incident schema: all six required fields must be there and not empty, the email must be valid, the category must be delay, collision, service-suspension or infrastructure-issue, termsAccepted must be true, and unknown fields are rejected. An unknown operation returns "Unknown operation" and an unknown incident_id returns "Incident not found". Every error response still has the original correlation_id.

III. correlation_id flow

- homework_demo.py creates the correlation_id one time (uuid4) and sends it in the HTTP request.
- The FastAPI wrapper does not make a new one. It puts the same value in the Kafka message (also as the message key) and waits on a future saved under that id.
- The worker reads the correlation_id from the request and copies it unchanged into its response, which it sends to the topic in reply_to.
- The wrapper's listener on s4098_entity_responses finds the waiting future with the same correlation_id and returns the response to the client. The client checks that the id is still the same.

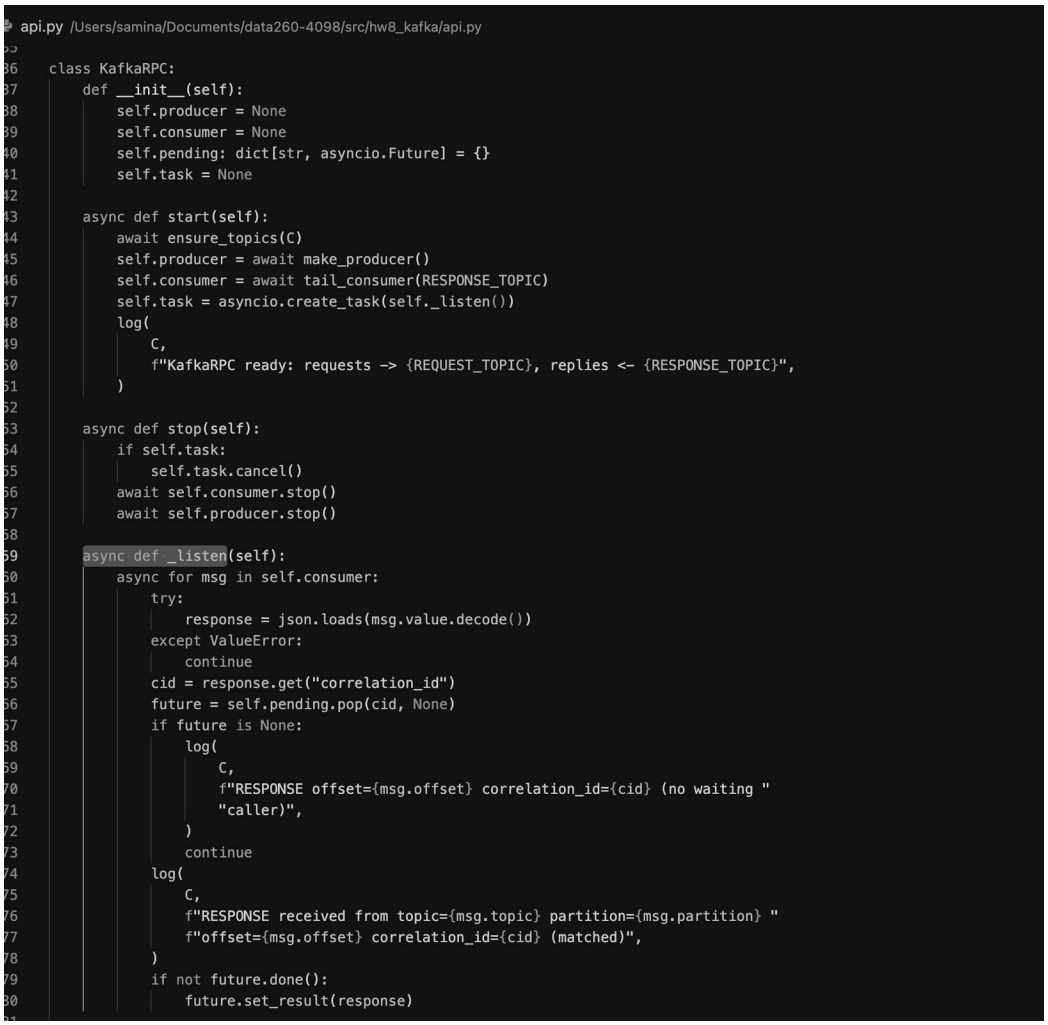
```
68
69  def call(step: str, operation: str, data: dict) -> tuple[str, dict]:
70      cid = str(uuid.uuid4())  # created once, here, by the client
71      clog(f"{step} correlation_id created: {cid}")
72      body = {"correlation_id": cid, "operation": operation, "data": data}
73      clog(
74          f"{step} SEND {operation} correlation_id={cid} via {API} -> Kafka topic "
75          f"{REQUEST_TOPIC} (reply_to={RESPONSE_TOPIC})"
76      )
77      clog(f"{step} request body={json.dumps(body)}")
78      response = post(body)
79      clog(f"{step} RESPONSE received by client: {json.dumps(response)}")
80      same = response.get("correlation_id") == cid
81      clog(f"{step} correlation_id preserved: {same} ({response.get('correlation_id')})")
82      if not same:
83          raise SystemExit("correlation_id changed on the way - this must not happen")
84      return cid, response
```

Figure 5. Client code (homework_demo.py): call() creates the correlation_id once, sends it with the operation and data to the FastAPI wrapper, and checks that the response carries the same correlation_id.



```
36
37  def short(value: dict) -> str:
38      return json.dumps(value, ensure_ascii=False)
39
40
41  async def main() -> None:
42      banner("Part 1 worker (python -m src.hw8_kafka.worker)")
43      idempotency = os.environ.get("HW8_IDEMPOTENCY", "on").lower() != "off"
44      service = IncidentService(idempotency=idempotency)
45      await ensure_topics(C)
46
47      consumer = AIOKafkaConsumer(
48          REQUEST_TOPIC,
49          bootstrap_servers=KAFKA_BOOTSTRAP,
50          group_id=WORKER_GROUP,
51          enable_auto_commit=False,  # commit only after the reply is written
52          auto_offset_reset="earliest",
53      )
54      producer = await make_producer()
55      await consumer.start()
56      log(
57          C,
58          f"group={WORKER_GROUP} topic={REQUEST_TOPIC} "
59          f"idempotency={'on' if idempotency else 'off'}",
60      )
61      log(C, "Consumer is ready and listening...")
62
63      stop = asyncio.Event()
64      loop = asyncio.get_running_loop()
65      for sig in (signal.SIGTERM, signal.SIGINT):
66          loop.add_signal_handler(sig, stop.set)
67
68      try:
69          while not stop.is_set():
70              batch = await consumer.getmany(timeout_ms=500)
71              for tp, messages in batch.items():
72                  for msg in messages:
73                      await handle_message(consumer, producer, service, msg)
74                      # commit exactly this message (not the whole fetched batch),
75                      # and only after its reply has been written
76                      await consumer.commit({tp: msg.offset + 1})
77      finally:
78          await consumer.stop()
79          await producer.stop()
80          log(C, "stopped")
81
```

Figure 6. Worker code (src/hw8_kafka/worker.py): consumer on s4098_entity_requests with auto-commit off; each message is handled (validate, CREATE/GET, reply to reply_to with the same correlation_id), then exactly that message's offset is committed after the reply is written.



```python
class KafkaRPC:
    def __init__(self):
        self.producer = None
        self.consumer = None
        self.pending: dict[str, asyncio.Future] = {}
        self.task = None

    async def start(self):
        await ensure_topics(C)
        self.producer = await make_producer()
        self.consumer = await tail_consumer(RESPONSE_TOPIC)
        self.task = asyncio.create_task(self._listen())
        log(
            C,
            f"KafkaRPC ready: requests -> {REQUEST_TOPIC}, replies <- {RESPONSE_TOPIC}",
        )

    async def stop(self):
        if self.task:
            self.task.cancel()
        await self.consumer.stop()
        await self.producer.stop()

    async def _listen(self):
        async for msg in self.consumer:
            try:
                response = json.loads(msg.value.decode())
            except ValueError:
                continue
            cid = response.get("correlation_id")
            future = self.pending.pop(cid, None)
            if future is None:
                log(
                    C,
                    f"RESPONSE offset={msg.offset} correlation_id={cid} (no waiting "
                    "caller)",
                )
                continue
            log(
                C,
                f"RESPONSE received from topic={msg.topic} partition={msg.partition} "
                f"offset={msg.offset} correlation_id={cid} (matched)",
            )
            if not future.done():
                future.set_result(response)
```

Figure 7. FastAPI wrapper (src/hw8_kafka/api.py): KafkaRPC.start() opens the producer and a listener on s4098_entity_responses; _listen() completes the waiting future whose correlation_id matches.

```
                future.set_result(response)

    async def request(self, operation: str, data: Any, correlation_id: str) -> dict:
        message = {
            "correlation_id": correlation_id,
            "reply_to": RESPONSE_TOPIC,
            "operation": operation,
            "data": data,
        }
        future = asyncio.get_running_loop().create_future()
        self.pending[correlation_id] = future
        meta = await self.producer.send_and_wait(
            REQUEST_TOPIC, encode(message), key=correlation_id.encode()
        )
        log(
            C,
            f"SEND to topic={meta.topic} partition={meta.partition} "
            f"offset={meta.offset} "
            f"correlation_id={correlation_id}",
        )
        log(C, f"SEND request={json.dumps(message)}")
        try:
            response = await asyncio.wait_for(future, timeout=TIMEOUT_SECONDS)
        except asyncio.TimeoutError:
            self.pending.pop(correlation_id, None)
            log(C, f"TIMEOUT correlation_id={correlation_id}")
            return {
                "correlation_id": correlation_id,
                "success": False,
                "error": f"No response from worker within {TIMEOUT_SECONDS}s",
            }
        log(C, f"RETURN HTTP response to caller correlation_id={correlation_id}")
        return response


rpc = KafkaRPC()
```

Figure 8. FastAPI wrapper: request() builds the Kafka message (correlation_id, reply_to, operation, data), saves the future, sends it to s4098_entity_requests and waits for the matching reply (15 s timeout).

IV. Demonstration (homework_demo.py)

homework_demo.py runs the full flow: create a correlation_id, send CREATE_INCIDENT, take the generated incident_id from the response, send GET_INCIDENT with it, and show the retrieved incident. At the end it merges its own log with the worker and API container logs, so one excerpt shows every step in order.

My first run of the demo returned HTTP 404. The reason was that my old HW7 API was still running on my Mac on the same port 8498 and answered before Docker. I found the process with `lsof -iTCP:8498`, stopped it, and after that the demo worked. The screenshots below are from the final run.

```
=== Steps 1–7: CREATE_INCIDENT ===
2026-10-30T03:30:17.141Z [CLIENT] [CREATE] correlation_id created: 18afa51f-be37-4fe7-bfb6-7cf0c78b39dc
2026-10-30T03:30:17.141Z [CLIENT] [CREATE] SEND CREATE_INCIDENT correlation_id=18afa51f-be37-4fe7-bfb6-7cf0c78b39dc via http://127.0.0.1:8498/incidents/kafka -> Kafka topic s4098_entity_requests (reply_to=s4098_entity_responses)
2026-10-30T03:30:17.141Z [CLIENT] [CREATE] request body={"correlation_id": "18afa51f-be37-4fe7-bfb6-7cf0c78b39dc", "operation": "CREATE_INCIDENT", "data": {"incidentTitle": "Bus collision at Diridon Station", "routeLine": "VTA Route 22", "submitterEmail": "kazisaminamaraj.mumu@sjsu.edu", "description": "A Route 22 bus hit a parked car while leaving Diridon Station; no injuries, the bus was taken out of service.", "category": "collision", "termsAccepted": true, "passengersAffected": 18}}
2026-10-30T03:30:17.160Z [CLIENT] [CREATE] RESPONSE received by client: {"correlation_id": "18afa51f-be37-4fe7-bfb6-7cf0c78b39dc", "success": true, "operation": "CREATE_INCIDENT", "incident_id": "2ebed720-7589-4b04-a1d1-977edbf1eafc", "message": "Incident created"}
2026-10-30T03:30:17.160Z [CLIENT] [CREATE] correlation_id preserved: True (18afa51f-be37-4fe7-bfb6-7cf0c78b39dc)
2026-10-30T03:30:17.160Z [CLIENT] [CREATE] extracted generated incident_id=2ebed720-7589-4b04-a1d1-977edbf1eafc (correlation_id=18afa51f-be37-4fe7-bfb6-7cf0c78b39dc)
((.venv) ) Kazi Samina Maraj Mumu data260-4098 %
```

Figure 9. homework_demo.py, steps 1-7: the client creates correlation_id 18afa51f..., sends CREATE_INCIDENT with the incident data through FastAPI to s4098_entity_requests, receives success=true with the generated incident_id 2ebed720..., and the correlation_id is preserved.

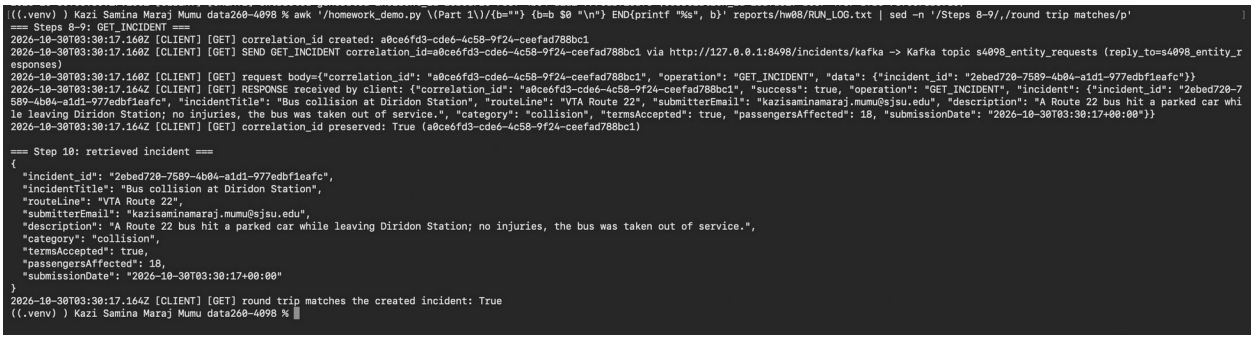


Figure 10. homework_demo.py, steps 8-10: GET_INCIDENT (correlation_id a0ce6fd3...) with the extracted incident_id 2ebed720... returns the stored incident with every field; correlation_id preserved and the round trip matches.



Figure 11. Full message flow (client + API + worker logs). CREATE_INCIDENT (18afa51f...): client -> API sends to s4098_entity_requests (offset 2) -> worker receives and processes -> reply to s4098_entity_responses (offset 2) -> API matches -> client gets incident_id 2ebed720.... Then GET_INCIDENT (a0ce6fd3..., offset 3) returns the same incident. The correlation_id never changes.

Part 2 - Three-Agent Kafka Pipeline

I. Topics and workflow

Sender	Reads topic	Sends to
send_question.py (me)	-	s4098_inbox (a question)
Planner	s4098_inbox	s4098_tasks (a plan with 3-4 steps)
Writer	s4098_tasks (2 partitions)	s4098_drafts (a draft answer)
Reviewer	s4098_drafts	s4098_final (status + answer)

Each agent is its own process in its own terminal with its own consumer group (s4098_planners, s4098_writers, s4098_reviewers). No agent imports or calls another agent; they only share the

topic names from `src/hw8_kafka/config.py`. All three use the same Kafka loop in `src/hw8_kafka/agent_runtime.py` and only differ in what they ask qwen3:8b (through my existing `src/model_client.py`). Every agent copies the correlation_id unchanged, uses it as the message key, adds one trace entry (topic, partition, offset, times) and commits its input offset only after its output was sent.

src/hw8_kafka/agents.py - Writer (short excerpt)

```
async def write(message: dict, ctx) -> dict:
    question, steps = check_task(message)          # a bad plan -> PoisonMessage
    numbered = "\n".join(f"{i}. {s}" for i, s in enumerate(steps, 1))
    draft = await _chat("You are the Writer ... at most 150 words.",
                        f"Question: {question}\nPlan:\n{numbered}", json_mode=False)
    return {"question": question, "plan": message["plan"], "draft": draft.strip()}
```

src/hw8_kafka/agent_runtime.py - what every agent does with one message (short)

```
output = await self.handler(message, self.ctx)          # Planner / Writer / Reviewer
output["correlation_id"] = cid                            # copied unchanged
output["trace"] = message.get("trace", []) + [{"agent": self.name, "in_topic": msg.topic,
                  "partition": msg.partition, "offset": msg.offset, ...}]
await self.producer.send_and_wait(self.out_topic, encode(output), key=cid.encode())
await self.commit(msg)                                    # input offset committed last
```

II. One complete message journey

I started the Planner, Writer and Reviewer in three terminals and then sent one question with send_question.py: "A light-rail train lost power and stopped between two stations during the evening rush. What should the transit agency do in the first 30 minutes?"



Figure 12. Planner: receives the question from s4098_inbox (offset 0, correlation_id d2ca2b39...), creates a plan with qwen3:8b (about 5 s), sends it to s4098_tasks and only then commits offset 1.

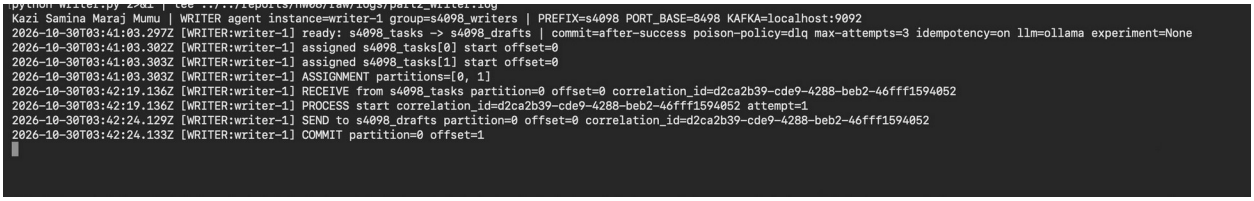


Figure 13. Writer: owns both partitions of s4098_tasks; receives the plan (partition 0, offset 0) with the same correlation_id, writes the draft with qwen3:8b (about 5 s), sends it to s4098_drafts and then commits.

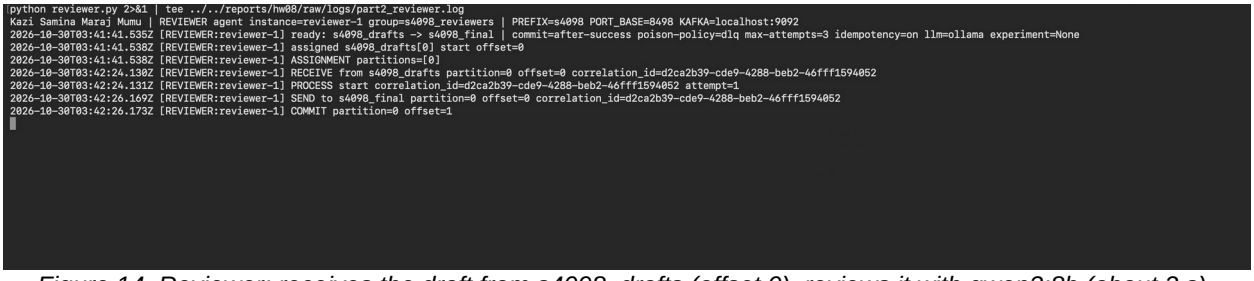


Figure 14. Reviewer: receives the draft from s4098_drafts (offset 0), reviews it with qwen3:8b (about 2 s), sends the approval to s4098_final and then commits.

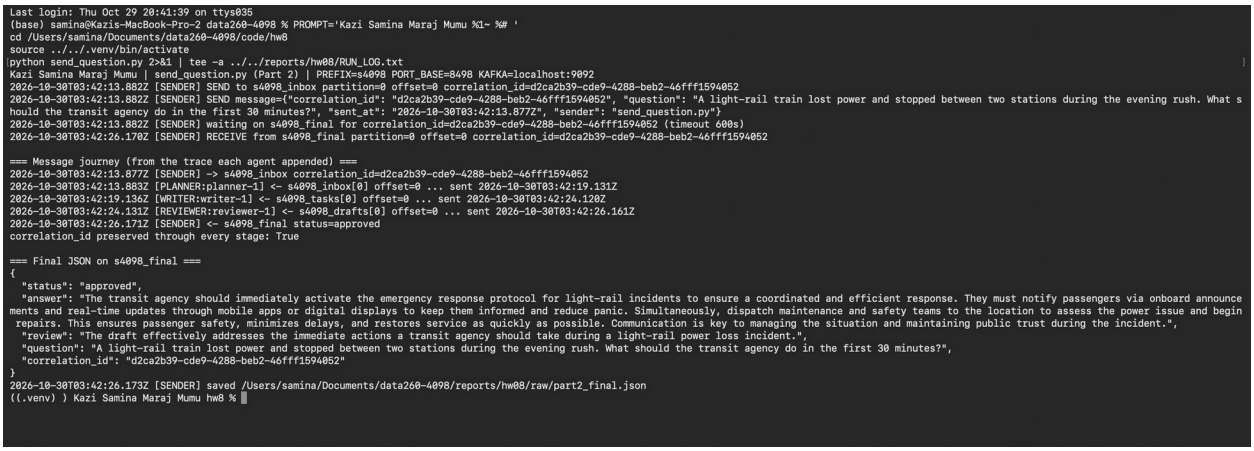


Figure 15. send_question.py: the journey shows every stage with topic, partition, offset and time; the correlation_id is preserved; the final JSON on s4098_final has status "approved" and the answer.

Stage	Read from	Offset	Time in stage
Planner	s4098_inbox[0]	0	5.25 s
Writer	s4098_tasks[0]	0	4.98 s
Reviewer	s4098_drafts[0]	0	2.03 s
Total (send -> final received)			12.3 s

Final message on s4098_final (correlation_id d2ca2b39-cde9-4288-beb2-46fff1594052)


```
{ "status": "approved",
  "answer": "The transit agency should immediately activate the emergency response protocol
     for light-rail incidents ... notify passengers via onboard announcements and real-time
     updates ... dispatch maintenance and safety teams to the location ...",
  "review": "The draft effectively addresses the immediate actions a transit agency should
     take during a light-rail power loss incident.",
  "correlation_id": "d2ca2b39-cde9-4288-beb2-46fff1594052" }
```


III. How Kafka helped my agents coordinate

Kafka was the only connection between the three agents, so each topic worked like a contract. The Planner only had to put a valid plan on s4098_tasks; it did not need to know who reads it or whether the Writer was up. Because messages stay in the topic, an agent that is slow or started later does not lose anything; it just continues from its committed offset. The correlation_id and the trace in every message let me follow one question through all three agents and see exactly how long each stage took (Planner 5.25 s, Writer 4.98 s, Reviewer 2.03 s), without any shared memory between the processes.

Part 3 - Failure Behavior

I ran every failure test with `code/hw8/run_failure_tests.py <test> --mode before|after`. The "before" mode turns one protection off with a flag; the "after" mode is my normal code. The runner starts its own Writer processes, prints their log lines inline, and saves the raw data in reports/hw08/raw/. Every run uses its own consumer group and an "experiment" tag on its messages, so test messages never reach my normal Part 2 pipeline.

I. Duplicate delivery

I replayed the exact same CREATE_INCIDENT message (same bytes, same correlation_id) to the Part 1 worker. Before the fix (worker started with HW8_IDEMPOTENCY=off) every delivery created a new incident. For the fix, the worker remembers the response of every finished CREATE by correlation_id. When the same correlation_id comes again, it sends the saved response back and does not create anything new.

src/hw8_kafka/incidents.py - idempotency

```
def handle(self, request):
    cid = request.get("correlation_id")
    if self.idempotency and cid in self.processed:
        return dict(self.processed[cid]), True        # replay: same answer, no new
incident
    response = {"correlation_id": cid, **self._process(request)}
    if self.idempotency and cid and request.get("operation") == "CREATE_INCIDENT":
        self.processed[cid] = response
    return response, False
```



Figure 16. Duplicate delivery, before the fix: the same CREATE_INCIDENT (correlation_id ef920dc3...) is delivered twice (offsets 4 and 5); the worker creates two different incidents (fb2be043... and 418ef4a7..., store size 1 -> 2).

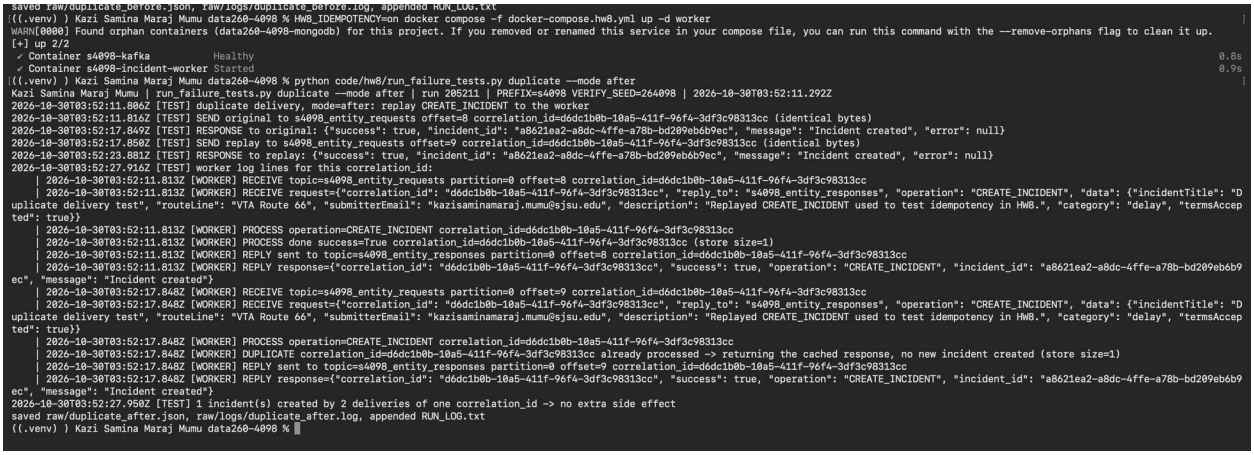


Figure 17. Duplicate delivery, after the fix: the same CREATE_INCIDENT (d6dc1b0b...) is delivered twice (offsets 8 and 9); the second time the worker logs DUPLICATE, returns the same incident_id a8621ea2... and the store size stays 1.

I did the same test on the Writer agent. There the side effect is a second draft.

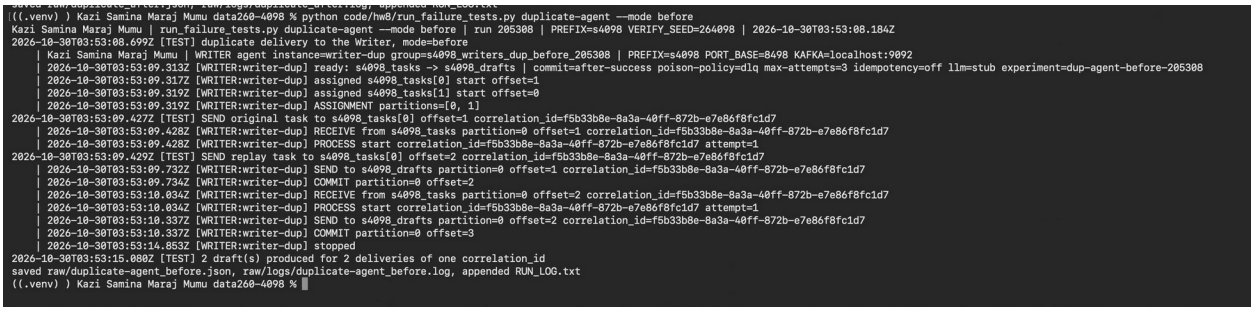


Figure 18. Writer, before the fix (idempotency off): the same task (f5b33b8e...) delivered twice gives 2 drafts on s4098_drafts.

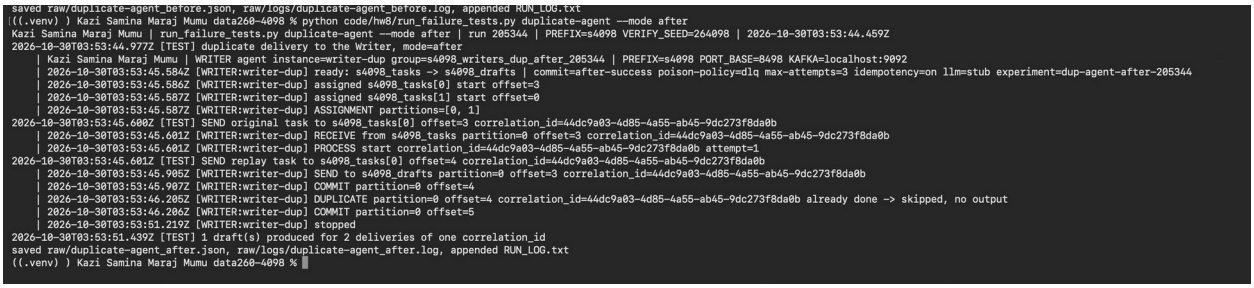


Figure 19. Writer, after the fix: the second delivery of task 44dc9a03... is skipped as DUPLICATE ("already done -> skipped, no output") and only its offset is committed: 1 draft.

II. Consumer death

The runner sends one task, waits until the Writer has started the qwen3:8b call, and then kills the Writer with kill -9 (2.6 s later). After that it restarts the Writer in the same consumer group. Before the fix the Writer used Kafka auto-commit (every 0.5 s), which commits the position of a message as soon as the code gets it, not when the work is done. After the fix auto-commit is off and the offset is committed only after the draft was written to s4098_drafts.

src/hw8_kafka/agent_runtime.py - commit only after success

```
consumer = AIOKafkaConsumer(..., enable_auto_commit=self.auto_commit)   # False normally
...
output = await self.handler(message, self.ctx)
meta = await self.producer.send_and_wait(self.out_topic, encode(output), key=key)
await self.commit(msg)     # consumer.commit({TopicPartition(msg.topic, msg.partition):
msg.offset + 1})
```

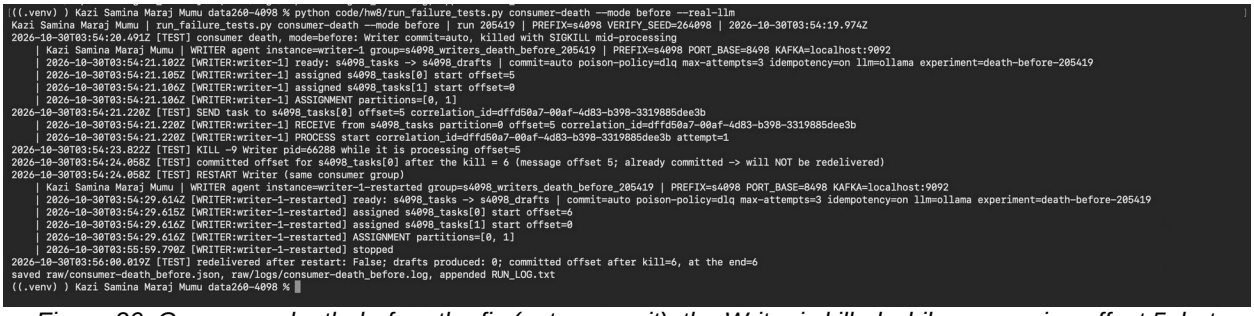


Figure 20. Consumer death, before the fix (auto-commit): the Writer is killed while processing offset 5, but offset 6 was already committed. The restarted Writer starts at offset 6, so the task is never redelivered: 0 drafts, the task is lost.

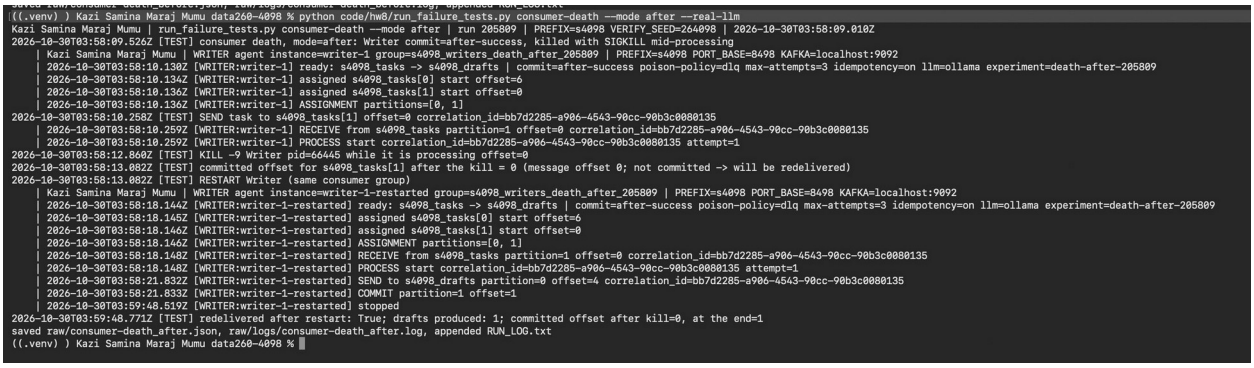


Figure 21. Consumer death, after the fix (commit after success): the Writer is killed while processing partition 1, offset 0; the committed offset is still 0, so Kafka redelivers the same message to the restarted Writer, which finishes it and only then commits offset 1: 1 draft, nothing lost.

I did not only trust my own script here. I also asked Kafka itself with the Kafka CLI, and the committed offset matches.

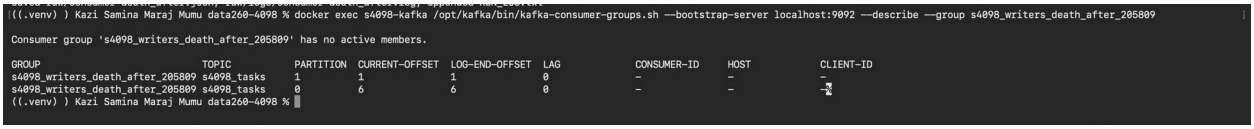


Figure 22. Kafka CLI check (kafka-consumer-groups.sh --describe) for group s4098_writers_death_after_205809: partition 1 has CURRENT-OFFSET 1 = LOG-END-OFFSET 1, lag 0, the same as the runner reported.

III. Poison message

A poison message here is a task whose plan is broken ("<<corrupted plan>>"), so the Writer can never process it. I sent it first and then a good task with the same key, so both are in the same partition. Before the fix the Writer retries forever. After the fix it tries at most 3 times with backoff (0.5 s, then 1 s), then writes the original message, the error, the source offset and the number of attempts to `s4098_dead_letter`, commits the offset and continues.

src/hw8_kafka/agent_runtime.py - bounded retries + dead-letter

```
except Exception as exc:
    if self.args.poison_policy == "block":       # before: no limit
        await asyncio.sleep(self.args.backoff); continue
    if attempt >= self.args.max_attempts:          # after: 3 attempts
        await self.dead_letter(msg, message, error, attempt)   # -> s4098_dead_letter
        await self.commit(msg)                     # the partition is free again
        return
    await asyncio.sleep(self.args.backoff * 2 ** (attempt - 1))
```

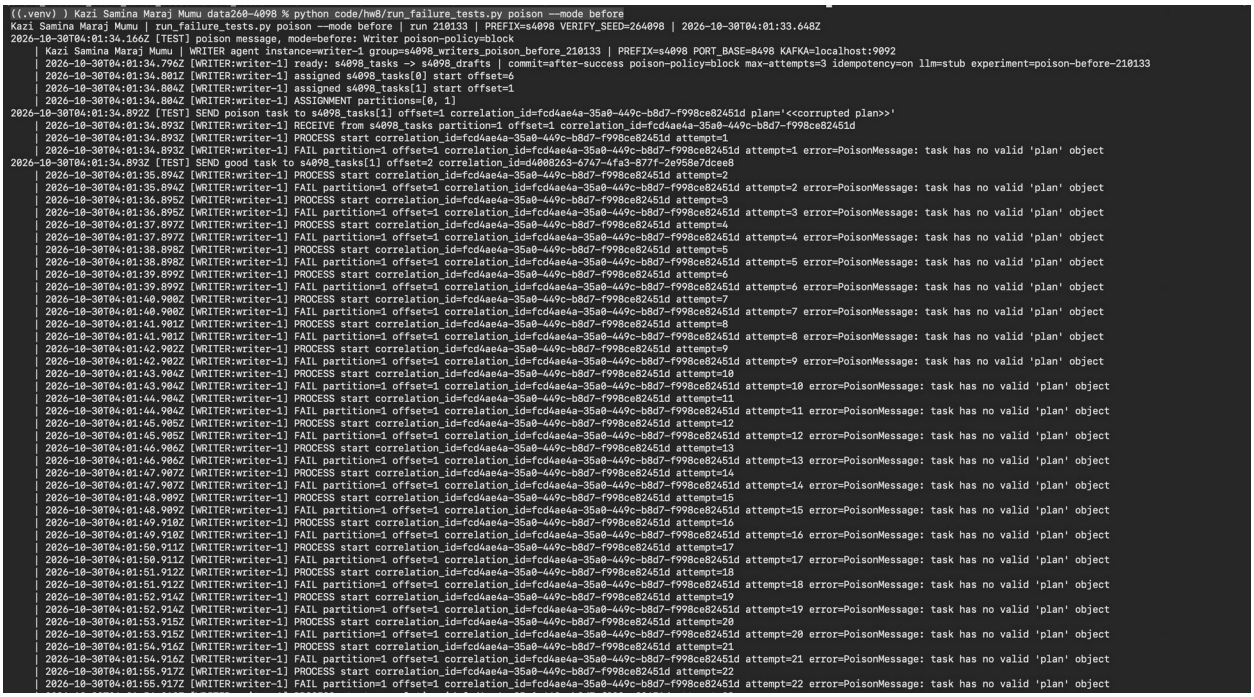


Figure 23. Poison message, before the fix: the corrupted task (partition 1, offset 1) fails and is retried every second without a limit; the good task at offset 2 is stuck behind it.



Figure 24. Poison message, before the fix (end): still failing after 27 attempts. Not dead-lettered and the good message was never processed. (The summary counts 17 attempts because it only counts the 15-second test window; the Writer kept retrying until the test stopped it.)

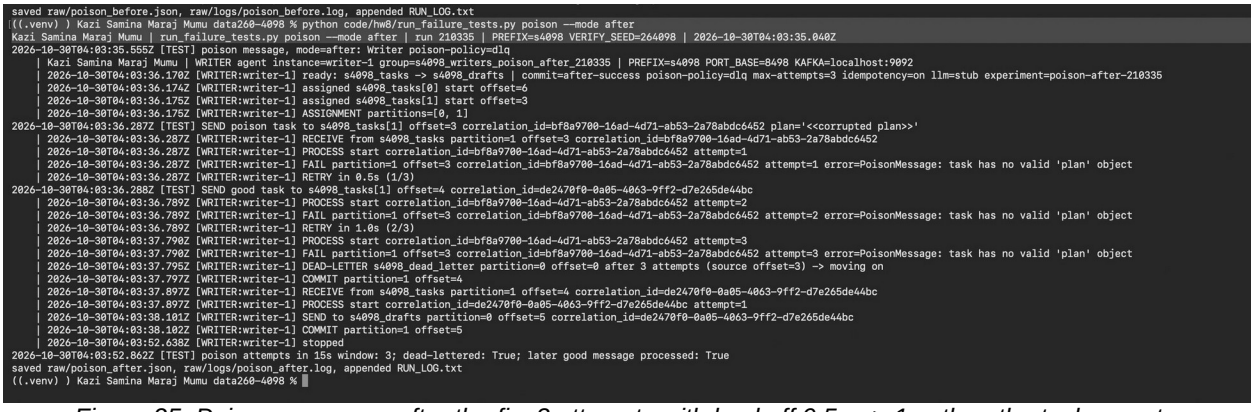


Figure 25. Poison message, after the fix: 3 attempts with backoff 0.5 s -> 1 s, then the task goes to s4098_dead_letter and its offset is committed; the good task at offset 4 is processed right after.

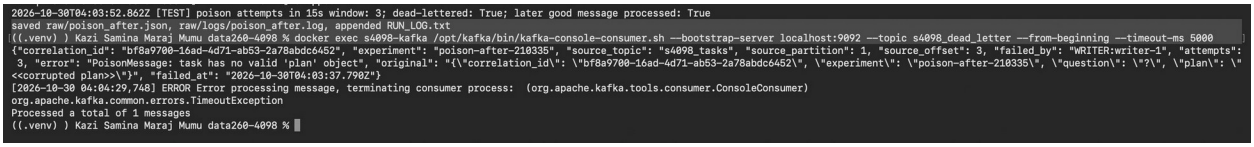


Figure 26. s4098_dead_letter read with the Kafka console consumer: the entry keeps the correlation_id, source topic/partition/offset, who failed it, the 3 attempts, the error and the original message. (The TimeoutException only marks the end of the 5 s read window.)

IV. Ordering

With VERIFY_SEED = 264098 the runner makes 20 related messages: 4 incidents with 5 status updates each (reported, crew_dispatched, crew_on_scene, service_restored, closed, so seq 1-5), mixed in a random order and each with a random work time. They are sent to s4098_tasks (2 partitions) and read by two Writer instances in the same consumer group, one partition each. In run 1 there is no key. In run 2 the key is the incident_id.

code/hw8/run_failure_tests.py - partition key

```
key = m["incident_id"].encode() if keyed else None      # run 2: key = incident_id
meta = await producer.send_and_wait(TASKS_TOPIC, encode(value), key=key)
```

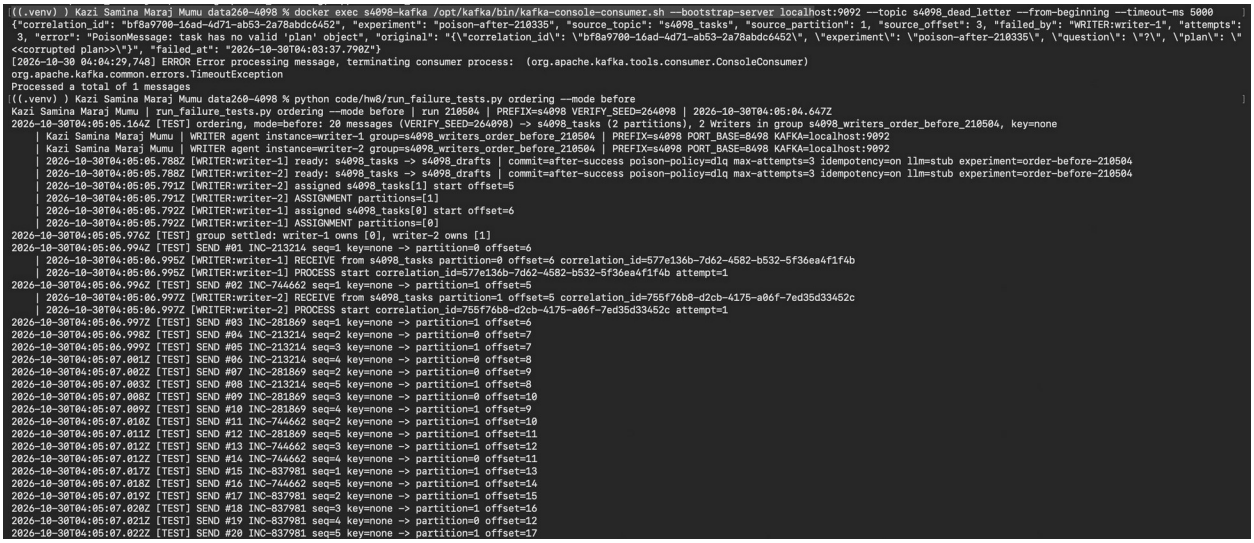


Figure 27. Ordering run 1 (no key): writer-1 owns partition 0 and writer-2 owns partition 1; the 20 VERIFY_SEED messages land in both partitions, so the updates of every incident are split.

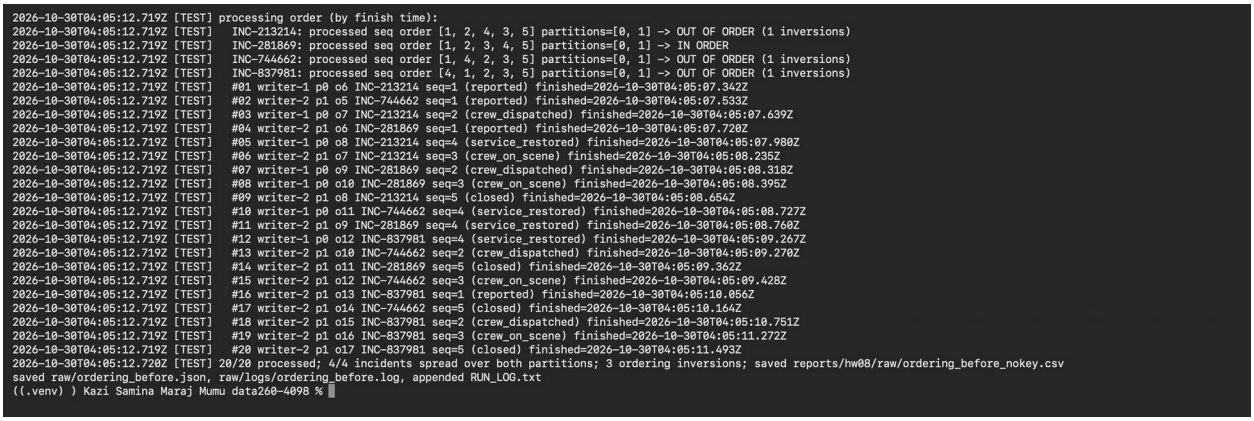


Figure 28. Ordering run 1 result: 3 of 4 incidents processed out of order (INC-213214: 1,2,4,3,5; INC-744662: 1,4,2,3,5; INC-837981: 4,1,2,3,5, so "service_restored" before "reported").

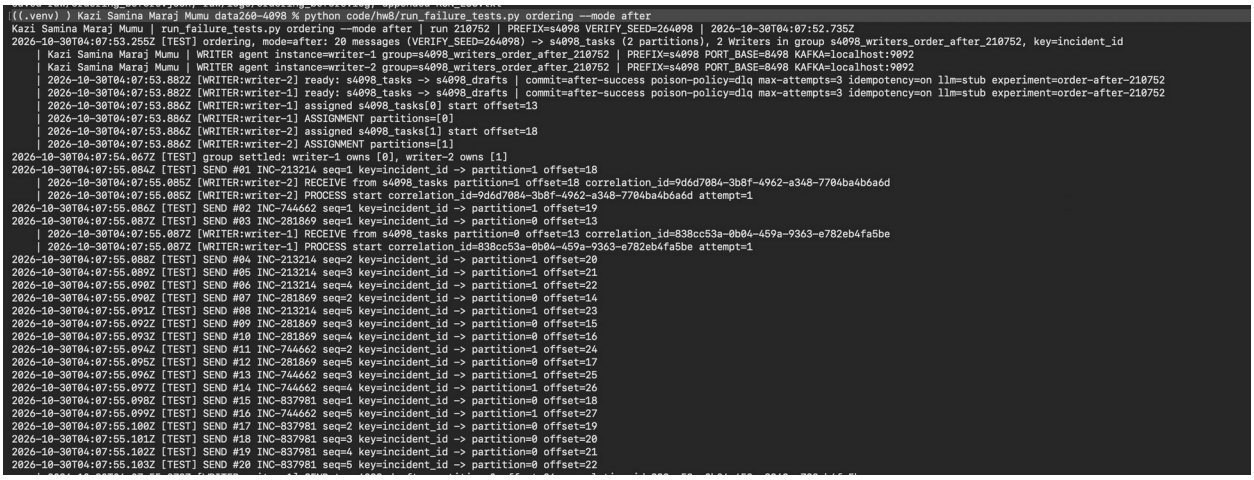


Figure 29. Ordering run 2 (key = incident_id): every update of one incident goes to the same partition (INC-213214 and INC-744662 -> 1; INC-281869 and INC-837981 -> 0).

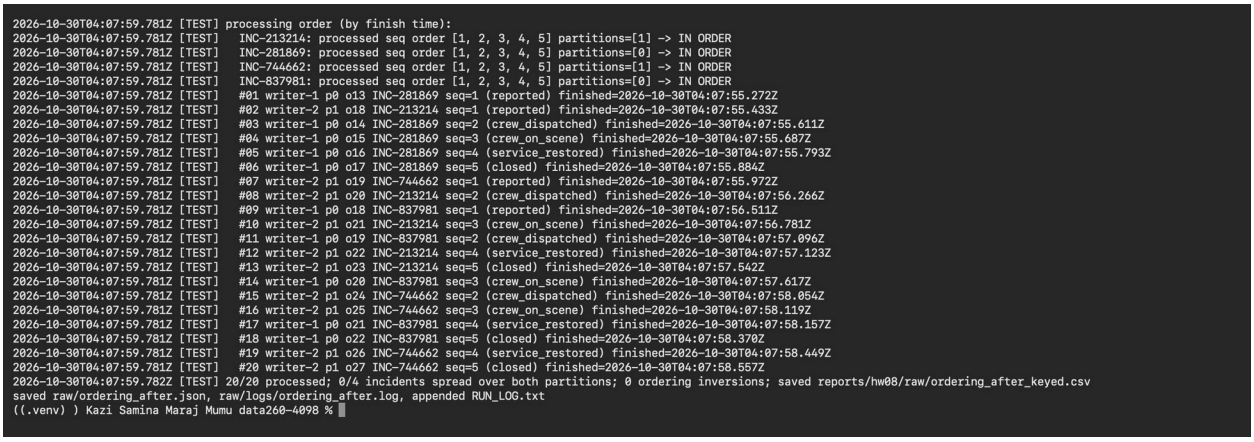


Figure 30. Ordering run 2 result: each incident stays in one partition and is processed by one Writer in offset order, so all 4 incidents are 1,2,3,4,5. The two Writers still work in parallel on different incidents.

Kafka only keeps order inside one partition. With no key, the two updates of one incident can sit in different partitions and two Writers can finish them in any order. With incident_id as the key, all updates of an incident are in one partition and keep their order, and different incidents can still run in parallel.

V. Failure table

Failure scenario	Before fix	After fix
Duplicate delivery (worker)	2 incidents created from 2 deliveries of one correlation_id (Fig. 16)	1 incident; replay gets the same incident_id, no side effect (Fig. 17)
Duplicate delivery (Writer)	2 drafts for 1 task (Fig. 18)	1 draft; replay skipped as DUPLICATE (Fig. 19)
Consumer killed mid-work	Offset already committed (6) -> not redelivered, 0 drafts, task lost (Fig. 20)	Offset not committed (0) -> redelivered, 1 draft, committed after (Fig. 21, 22)
Poison message	Retried forever (27 attempts), not dead-lettered, good message blocked (Fig. 23, 24)	3 attempts -> s4098_dead_letter, good message processed (Fig. 25, 26)
Two partitions/two consumers	No key: 4/4 incidents in both partitions, 3 out of order (Fig. 27, 28)	Key = incident_id: 1 partition per incident, 0 out of order (Fig. 29, 30)

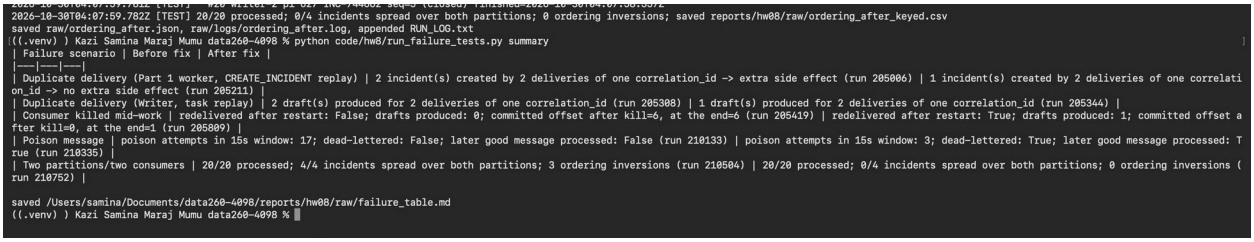


Figure 31. Failure table generated by the runner from reports/hw08/raw/failure_results.json (every row with its run id).

VI. Comparison with the HW2 in-process graph

In HW2 my LangGraph graph ran Planner, Reviewer and Finalizer as function calls in one process. Each step waited for the step before it and they all shared one state object.

Two benefits I got with Kafka:

- **Independent scaling.** In the ordering test I added a second Writer just by starting it in the same consumer group, and Kafka split the two partitions of s4098_tasks between them. In HW2 the only way to scale was to run the whole graph again.
- **Decoupled failure.** When the Writer was killed, the task waited in Kafka and was finished after the restart from the committed offset, and the other agents were not affected. In HW2 an exception in one node ended the whole run and the in-memory state was gone.

Two new responsibilities I now have to handle myself:

- **Duplicate delivery and offsets.** Kafka delivers at least once, so I had to decide when to commit (only after the output is written) and make processing idempotent by correlation_id. A function call in HW2 ran exactly once.
- **Ordering and bad messages.** Kafka keeps order only inside one partition, so I had to choose a message key (incident_id). And a message that always fails needs a retry limit and a dead-letter topic, otherwise it blocks its partition. In HW2 the order was fixed in the code and a bad input just raised an exception.

Verification

`make verify-hw08` is a smoke test on the committed code. It uses the running Docker stack, starts its own short-lived agents with a stub model (so it does not need Ollama), and checks behavior, not wording: the topics and partitions, the API CREATE/GET round trip, error responses with the same correlation_id, a direct Kafka round trip, the idempotent replay, the three-agent pipeline, retry + dead-letter, key -> partition with the VERIFY_SEED messages, and the offline tests. It writes reports/hw08/verification.json with the homework number, SID4, commit hash, model, SEED/VERIFY_SEED and every check. All 18 checks passed on commit d1c53f9.

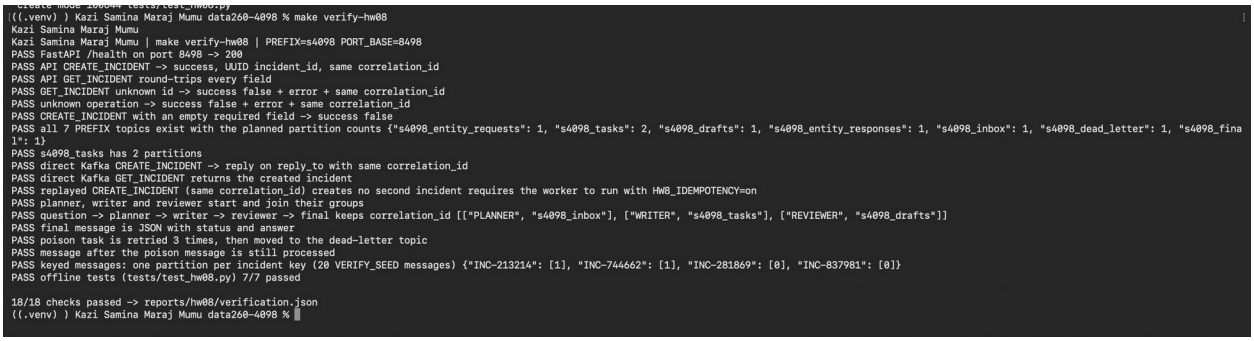


Figure 32. make verify-hw08: 18/18 checks passed, written to reports/hw08/verification.json.

AI Use

1. What did I use an AI assistant for, and what did I do myself?

I used Claude to read the HW8 assignment, the class demos and my repository, and to write the first version of the HW8 code: the Kafka worker and FastAPI wrapper for my Incident entity, homework_demo.py, the docker compose file, the Planner/Writer/Reviewer agents and their shared Kafka loop, the Part 3 test runner, the offline tests, verify_hw08.py and a run guide. Claude also helped me write this report from my screenshots and results. I ran everything myself on my Mac with the real apache/kafka:3.9.2 container and qwen3:8b: creating the topics, starting the containers, the demo, the four agent terminals, all Part 3 before/after runs and make verify-hw08. I took all screenshots, checked the consumer-death result with the Kafka CLI myself, and did the commit, tag and push.

2. One AI-produced output that was wrong, and one thing I verified myself.

Claude's first worker read messages in batches with getmany() and then called consumer.commit() with no arguments after the first message. That commits the position after the whole batch, so messages 2..n of a batch would have been marked as done before they were processed. This is exactly the "offset committed before processing" problem from Part 3 II. A second problem was in the first homework_demo.py: it sorted the merged client/API/worker log only by timestamp, and because several lines have the same millisecond, the API's "response received" line was printed before the worker's "reply sent" line, which looks impossible. I also verified one result myself instead of trusting the test script: for the consumer-death after-run I asked Kafka directly with kafka-consumer-groups.sh --describe for group s4098_writers_death_after_205809, and it showed CURRENT-OFFSET 1 = LOG-END-OFFSET 1 with lag 0 on partition 1, the same value the script reported "at the end".

3. How the problems were found / how I verified.

The batch-commit bug was found while reviewing the commit code against how aiokafka's commit() works without offsets (it commits the current position of every assigned partition). The log-order problem was visible in the first test output of the demo. For my own check I used the Kafka CLI on the real broker (Figure 22) and compared the committed offset with the message offset (0) and with the runner's value (1).

4. What was changed and why it works now.

Every consumer now commits exactly one message, commit({TopicPartition: msg.offset + 1}), and only after its output (response, plan, draft or final) was written with send_and_wait. In my consumer-death after-run the committed offset stayed at 0 after the kill, so the restarted Writer got the same message again and finished it; in the before-run (auto-commit) the offset was already 6 and the task was lost. The demo now orders the merged log by operation and by the

stage of the round trip (client send -> API send -> worker receive/process/reply -> API response -> client) and keeps the real timestamps, so the sequence in Figure 11 is in the right order.